03. Transformers 1: From Text to Attention

📌 The Fundamental Challenge: Text → Numbers

- Neural networks only operate on numbers — text means nothing to them directly
- We need a **translation layer** that converts words into numbers *while preserving meaning*
- The challenge: simple number assignment loses relationships (why is "cat" = 4 and "dog" = 5?)
- Three questions this lecture answers: how text becomes numbers, what the model "sees," and how attention works mechanically

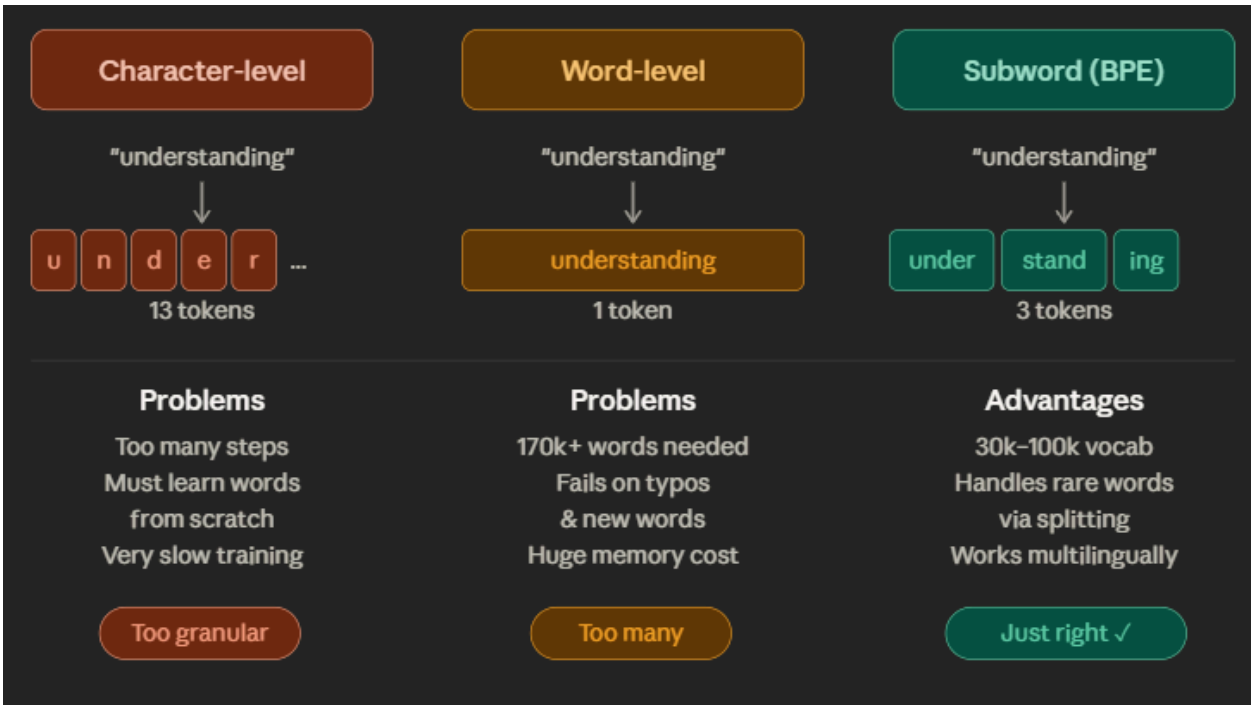
When a neural network processes language, it never actually "reads" — it performs matrix multiplications on floating-point numbers. Everything we do in NLP is fundamentally about building a bridge between human text and that mathematical world. The tricky part isn't just converting — it's converting in a way that keeps semantically similar things numerically similar.

Key takeaway: The gap between text and numbers is the core engineering problem that all of tokenization, embeddings, and positional encoding exist to solve.

📌 Tokenization: Breaking Text into Units

- **Tokenization** = splitting raw text into discrete units called **tokens**, then assigning each an integer ID
- Three approaches exist: character-level, word-level, and subword (the modern winner)
- The choice of granularity fundamentally shapes what the model can and can't learn

- Modern models (GPT, Claude) use **subword tokenization** with vocabularies of 30k–100k tokens



Character-Level

Every character gets its own integer ID. "Hello" → `[1, 2, 3, 3, 4]`. Simple to implement, but a word like "understanding" needs 13 separate processing steps. The model has to learn what a word even *is* before it can learn what it means — burning enormous amounts of data on basics.

> **Analogy:** Teaching someone to read by showing them individual letters and hoping they figure out that "c-a-t" means a furry animal. Technically possible. Wildly inefficient.

Word-Level

Each whole word gets an ID. "The cat sat" → `[1, 2, 3]`. Much more intuitive, but English alone has 170,000+ common words, pushing parameter counts into the billions just for the lookup table. Worse, any word not seen during training is completely invisible to the model — a problem called **Out-of-Vocabulary (OOV)**.

> **Example:** "Are 'cat', 'cats', and 'cat's' three different concepts?" — word-level says yes, requiring three separate entries for essentially the same thing.

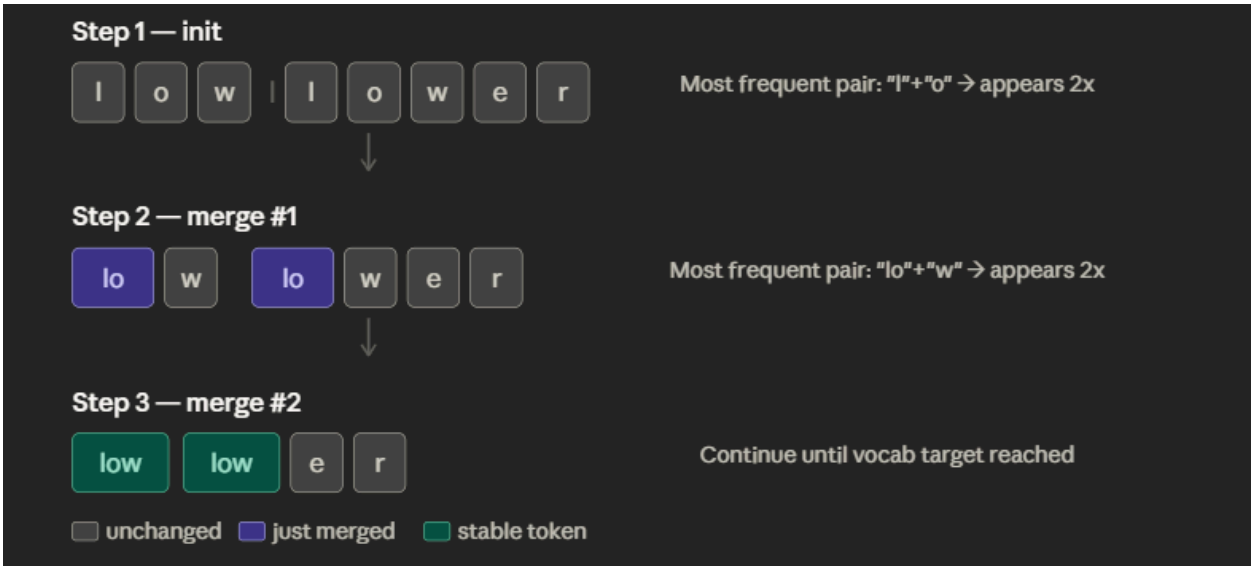
Subword (the solution)

Common words stay whole; rare or long words break into recognizable pieces. "understanding" → `["under", "stand", "ing"]`. A vocabulary of 30k–100k tokens can represent virtually any text in any language, because even a completely unknown word can be split into known subword fragments.

Key takeaway: Subword tokenization is the Goldilocks solution — not so granular that the model wastes effort on letters, not so coarse that it fails on new words.

📌 Byte Pair Encoding (BPE)

- **BPE** is the algorithm used to *build* a subword vocabulary from raw text
- Starts with individual characters, then iteratively merges the most frequent adjacent pairs
- Stops when the vocabulary reaches a target size (e.g., 50,000 tokens)
- Result: frequent patterns like "ing", "tion", "un" become single tokens; rare sequences stay split



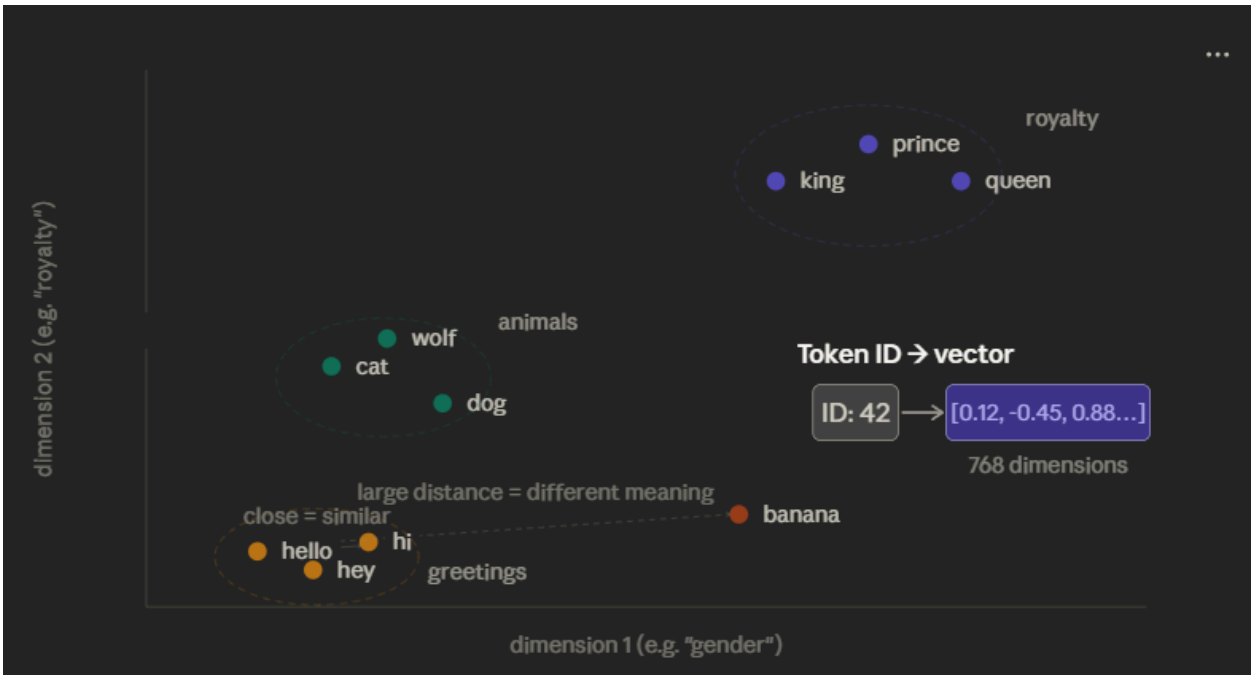
BPE is essentially a compression algorithm applied to vocabulary building. You start with a sea of individual characters and repeatedly combine the most common neighboring pair into a single new token. After enough iterations, common English words like "the", "ing", "tion" become single tokens, while rare or unknown words gracefully degrade into their constituent pieces. This means the model has never truly seen an "unknown" word — just a novel combination of known fragments.

> **Example:** The unknown word "Akwirw_ier" gets split into `["Ak", "w", "ir", "w", " ", "ier"]` — each piece has a known token ID. The model won't understand it, but it won't crash either.

Key takeaway: BPE is a self-organizing algorithm — it learns which units of language are most worth having as first-class tokens, purely from frequency statistics in training data.

📌 Embeddings: From IDs to Meaning

- Token IDs are arbitrary integers — `15496` doesn't "know" it means "Hello"
- **Embeddings** convert each token ID into a **dense vector** (a list of hundreds of numbers)
- Similar meanings → nearby vectors; distant meanings → distant vectors
- Each token gets a 768-dimensional vector (in base models), looked up from a large **embedding matrix**



The embedding matrix is a giant lookup table: row 42 contains the vector for token 42. The process is just indexing — no computation required. What makes it powerful is that these vectors were *learned* during training to cluster related concepts nearby in the high-dimensional space. The model didn't need to be told "cat and dog are similar" — it figured this out from seeing them used in similar contexts billions of times.

> **Example:** The classic demo — `king - man + woman ≈ queen`. This works because the vector offset from "man" to "woman" (roughly a "gender" direction) is the same as the offset from "king" to "queen." Relationships become geometric directions.

Important limitation: At this stage, embeddings are **context-free**. The word "bank" has exactly one vector, regardless of whether you mean a river bank or a financial institution. This is the problem that the attention mechanism will later solve.

Key takeaway: Embeddings transform discrete, meaningless token IDs into a rich geometric space where mathematical distance equals semantic similarity.

📌 The Problem of Sequence & Permutation Invariance

- Without extra information, attention treats token order as meaningless — this is called **permutation invariance**
- "The dog bit the man" and "The man bit the dog" contain identical tokens, just reordered
- Pure attention sees both as the same set of words and produces the same output
- We need to inject **positional information** — to make each token know both *what* it is and *where* it is

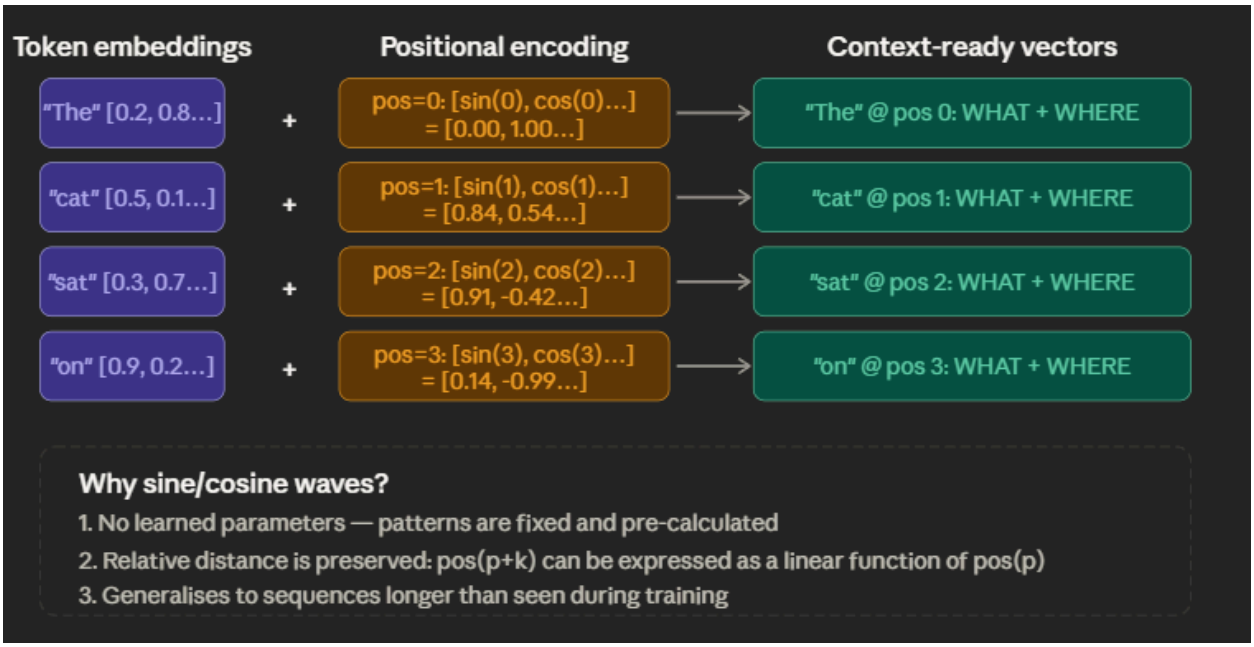
This is a fundamental problem with the transformer architecture's biggest strength. Because transformers process all tokens in parallel (unlike RNNs that process left-to-right), they lose the notion of sequence. The position of a word — whether it's the subject or object, the first word or the last — is critical for meaning. Without a fix, the model would be completely blind to grammar.

> **Analogy:** Imagine getting a bag of scrabble tiles and being asked to understand the sentence they formed. You have all the letters, but the order — the part that makes it a sentence — is gone.

Key takeaway: Permutation invariance is the price of parallelism, and positional encoding is the solution.

📌 Positional Encoding

- A **positional encoding** is a vector added to each token's embedding, representing its position in the sequence
- The original transformer uses **sine and cosine waves** at different frequencies for each dimension
- Formula: `PE(pos, 2i) = sin(pos / 10000^(2i/d_model))` and `PE(pos, 2i+1) = cos(...)`
- Every position gets a unique "fingerprint" combination of sine/cosine values



A small **position vector** is **added** to each word's embedding before it enters the network.

- Word at position 0 gets one pattern, position 1 gets another, etc.
- Each position gets a **unique fingerprint** of numbers
- This is created using **sine and cosine waves** of different frequencies

Formula (don't need to memorize, just understand):

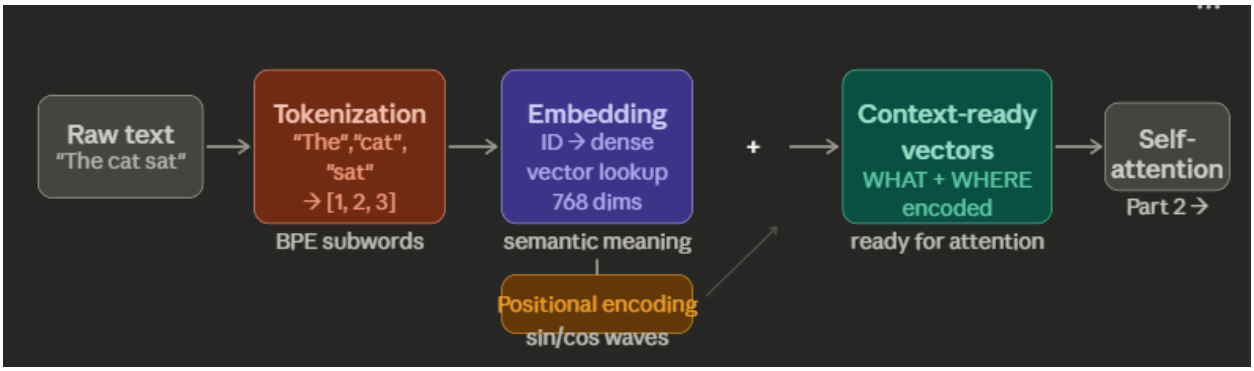
- Even dimensions → use sine wave
- Odd dimensions → use cosine wave

Why sine/cosine waves?

- No extra parameters to learn — patterns are pre-calculated
- Model can figure out *relative distance* between words
- Can handle sequences longer than what it trained on

The Full Input Pipeline

Here is everything covered in Part 1, assembled into a single end-to-end picture:



By the end of this pipeline, every token in your input sentence has been transformed into a rich vector that simultaneously encodes *what concept it represents* (from the embedding) and *where in the sentence it sits* (from the positional encoding). This combined vector is what enters the actual transformer layers — specifically, the self-attention mechanism that will be covered in Part 2.

Key takeaway: The three-step pipeline (tokenize → embed → positionally encode) transforms a raw string into a matrix of context-ready vectors that the transformer can finally reason over.

✅ Master Summary Table

Topic	What it is	Problem it solves	Key limitation
Character-level tokenization	One integer per character	Converts text to numbers	Too granular; 13 steps per word; model must learn vocabulary from scratch
Word-level tokenization	One integer per whole word	Intuitive concept mapping	170k+ vocab; OOV failures; 3B parameters just for lookup
Subword tokenization (BPE)	Frequent words whole; rare words split into pieces	Balances granularity vs. vocab size	Split points don't always match human intuition
BPE algorithm	Iteratively merges most-frequent adjacent token pairs	Learns optimal vocabulary from data automatically	Vocabulary is fixed after training; doesn't adapt to new domains

Topic	What it is	Problem it solves	Key limitation
Tokenization in practice	Spaces attach to next word; contractions split; rare words fragment	Compact, general-purpose text representation	Models can't count characters; non-English costs more tokens
Embeddings	Token ID → dense vector via lookup table	Captures semantic similarity as geometric proximity	Context-free: "bank" (river) = "bank" (finance) before attention
Semantic vector arithmetic	`king - man + woman ≈ queen`	Shows that relationships are encoded as directional offsets	Emergent, imperfect — doesn't work for all analogies
The problem of sequence	Attention is permutation-invariant by default	—	"The dog bit the man" = "The man bit the dog" without a fix
Positional encoding	Sine/cosine vectors added to embeddings	Injects word order (WHAT + WHERE) into every token	Original sinusoidal method struggles with very long contexts
Full input pipeline	Text → tokenize → embed → positionally encode → context-ready vectors	End-to-end text-to-numbers with meaning and order	Still no inter-word context — that's what self-attention adds (Part 2)